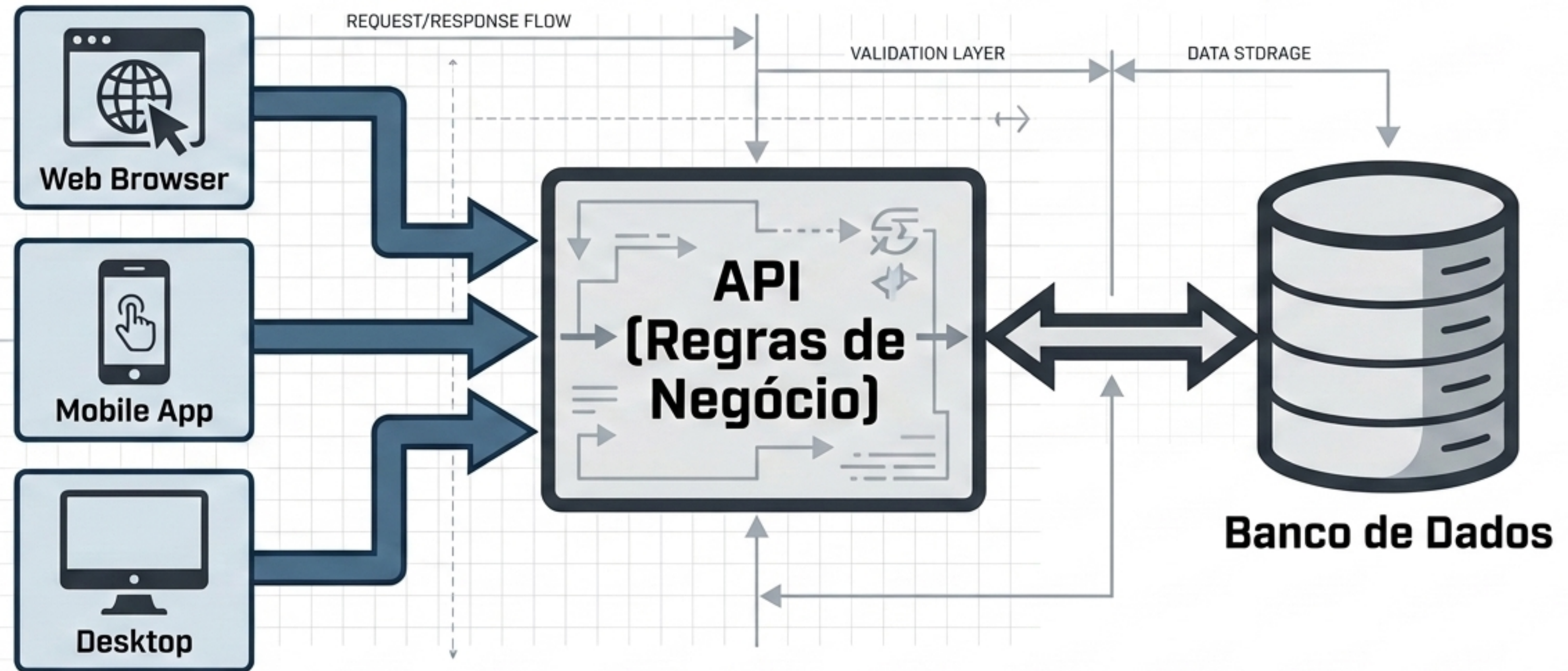


Aula 5: API REST

Prof. Ronaldo Borges

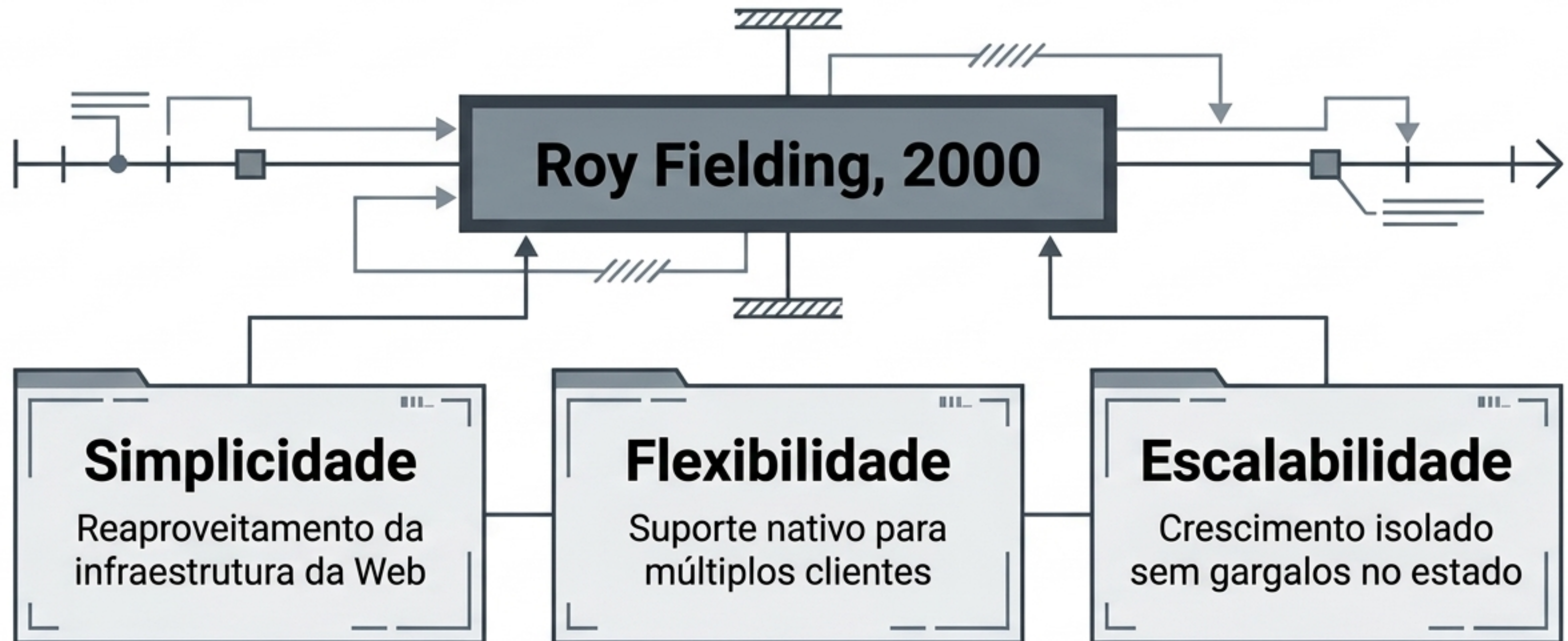
A Ponte da Integração

API = Application Programming Interface. **O que é uma API?** Sistema de back-end que centraliza a m de dados, garantindo validações e regras de negócio antes de atingir o banco de dados.



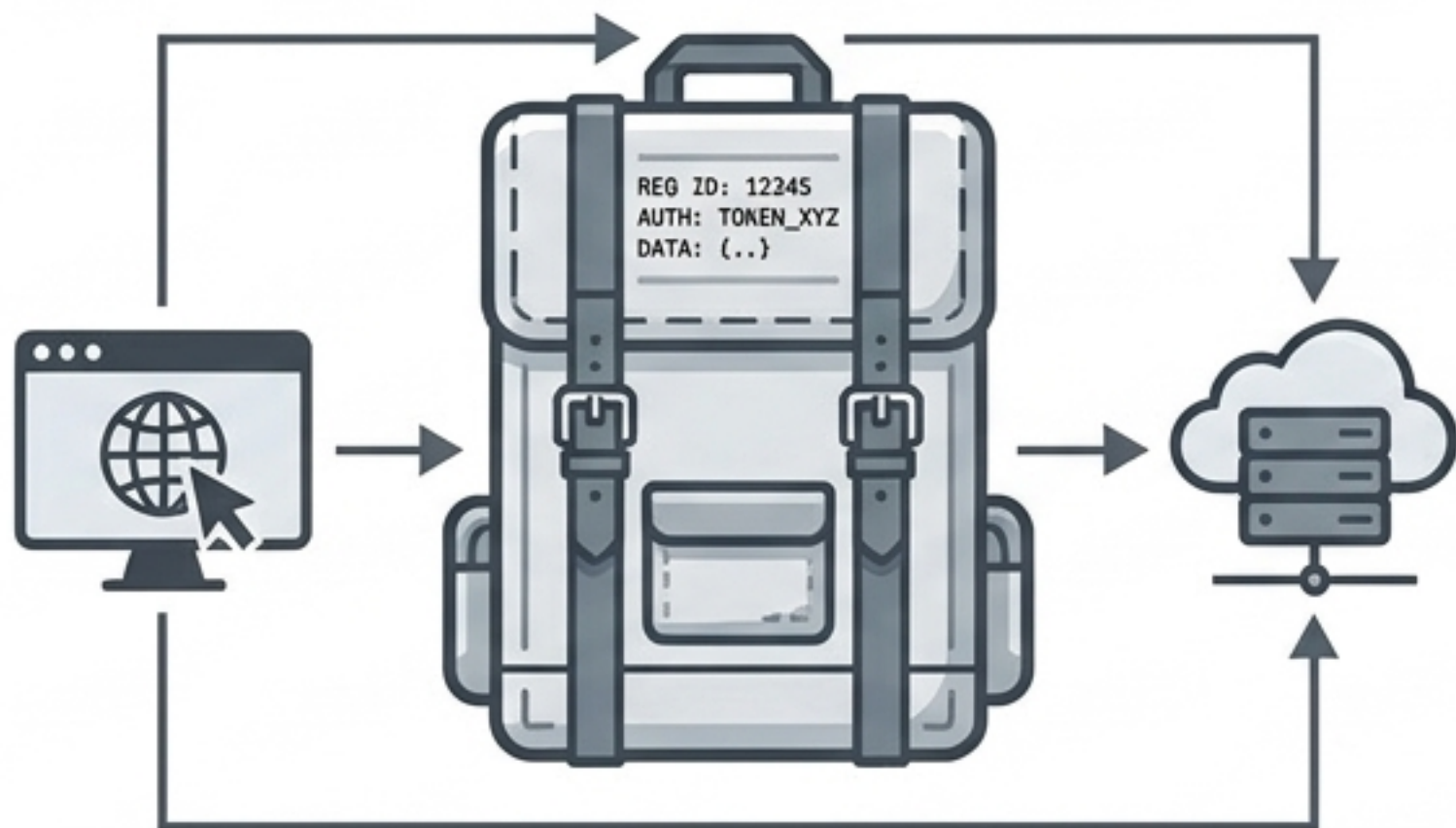
A Arquitetura REST (REpresentational State Transfer)

REST não é uma linguagem ou biblioteca. É um estilo arquitetural introduzido por Roy Fielding que padroniza serviços web.



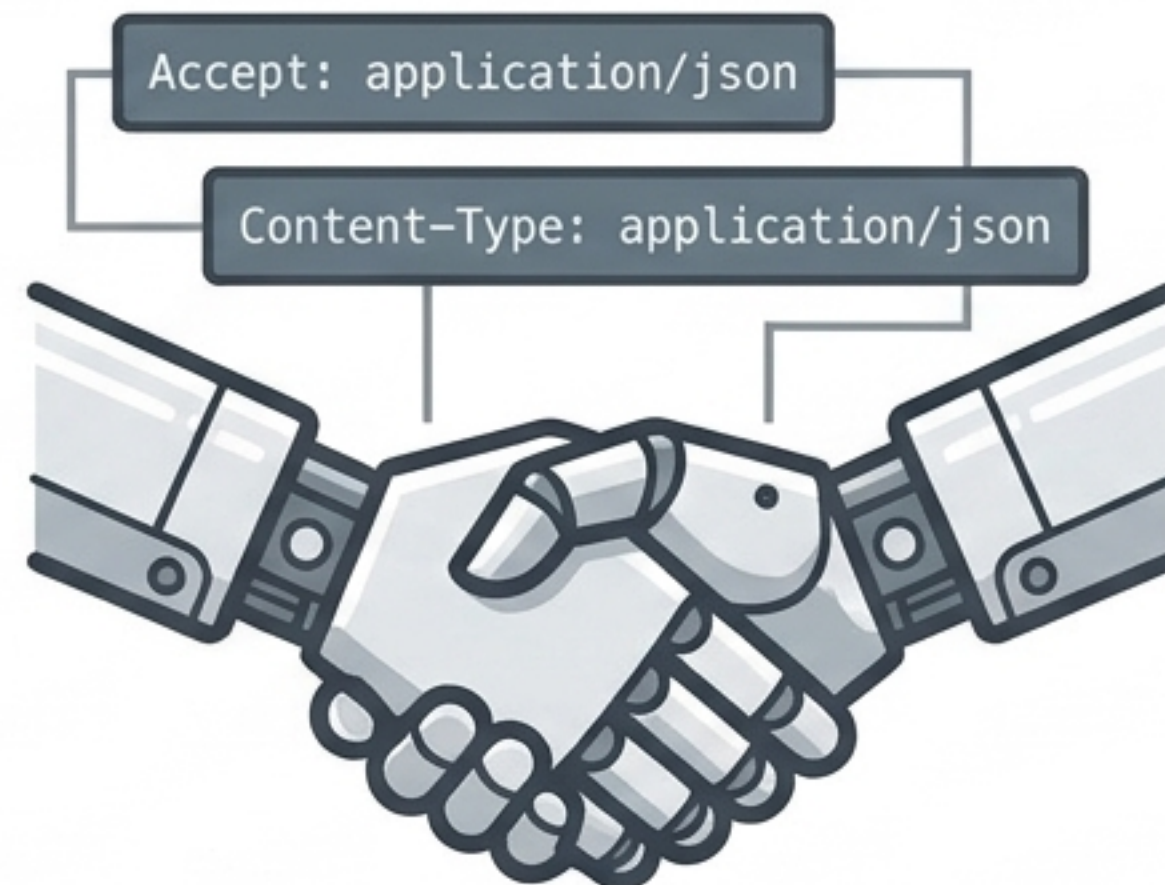
Princípios Fundamentais

Comunicação Stateless



O servidor não guarda memória (estado) das requisições anteriores. Cada requisição é um pacote independente e completo.

Negociação de Conteúdo



Troca de dados em formatos estruturados baseada estritamente nos cabeçalhos HTTP.

O Princípio HATEOAS: Navegação Dinâmica

Uma API sem HATEOAS é um beco sem saída. Com HATEOAS, ela entrega o mapa e as direções para o próximo passo.

HATEOAS = Hypermedia as the Engine of Application State

Onde ir

```
{
  "id": "12345",
  "status": "Pendente",
  "links": [
    {
      "rel": "atualizar",
      "href": "https://api.loja.com/pedidos/12345",
      "method": "PUT"
    },
    {
      "rel": "pagamento",
      "href": "https://api.loja.com/pedidos/12345/pagamento",
      "method": "POST"
    }
  ]
}
```

Como interagir

A Sintaxe das URIs: Mapeando Recursos

Regra de Ouro: Foque em Substantivos (recursos), nunca em Verbos (ações).
As ações já são definidas organicamente pelo método HTTP.

 **Faça isso**

/users

/produtos

Substantivos no plural representam corretamente a coleção.

 **Não faça isso**

~~/getUsers~~

~~/deletarProduto~~

Redundante e conflitante com os verbos naturais do protocolo HTTP.

Matriz de Operações

Verbo HTTP	Operação CRUD	Alvo	Exemplo de Rota
GET	Read (Leitura)	Coleção ou Item	/users ou /users/1
POST	Create (Criação)	Coleção	/users
PUT	Update (Atualização)	Item Específico	/users/1
DELETE	Delete (Remoção)	Item Específico	/users/1

O Idioma do Servidor: Status Codes



Sucesso

- **200 OK:** Requisição bem-sucedida (padrão para GET, PUT, DELETE).
- **201 Created:** Recurso criado com sucesso (padrão para POST).



Erros do Cliente

- **400 Bad Request:** Sintaxe inválida ou payload incorreto.
- **404 Not Found:** O endpoint ou o recurso (ex: ID) não existe.

Dissecando a Requisição (Node.js + Express)

api.loja.com/users/**1**?active=true

req.params

Extraí o '1' da URL. Parâmetros obrigatórios definidos na rota (/users/:id). Servem para identificar recursos.

req.query

Extraí '?active=true'. Filtros opcionais anexados ao final da URL.

req.body

(Oculto da URL). O 'payload' JSON contendo os dados complexos enviados (ex: num POST ou PUT).

Prática: O Setup do CRUD em Memória



```
const express = require('express');
const app = express();

app.use(express.json());

const users = [];

app.listen(8080, () => {
  console.log("Servidor on-line!");
});
```

Middleware obrigatório:

Ensina o Express a interpretar pacotes JSON no req.body

Simulando nosso Banco de Dados localmente no array JavaScript

Implementando CREATE e READ (POST / GET)

```
app.post('/users', (req, res) => {  
  const user = req.body;  
  users.push(user);  
  res.status(201).send('Usuário criado com sucesso!');  
});
```

Retorna 201 Created
confirmando a gravação.

```
app.get('/users', (req, res) => {  
  res.json(users);  
});
```

Retorna automaticamente
200 OK e converte o array
para JSON.

Implementando UPDATE e DELETE (PUT / DELETE)

Atenção: Note a exigência do `/:id` na rota e o tratamento crítico para devolver status-`(404)` quando a busca falha.

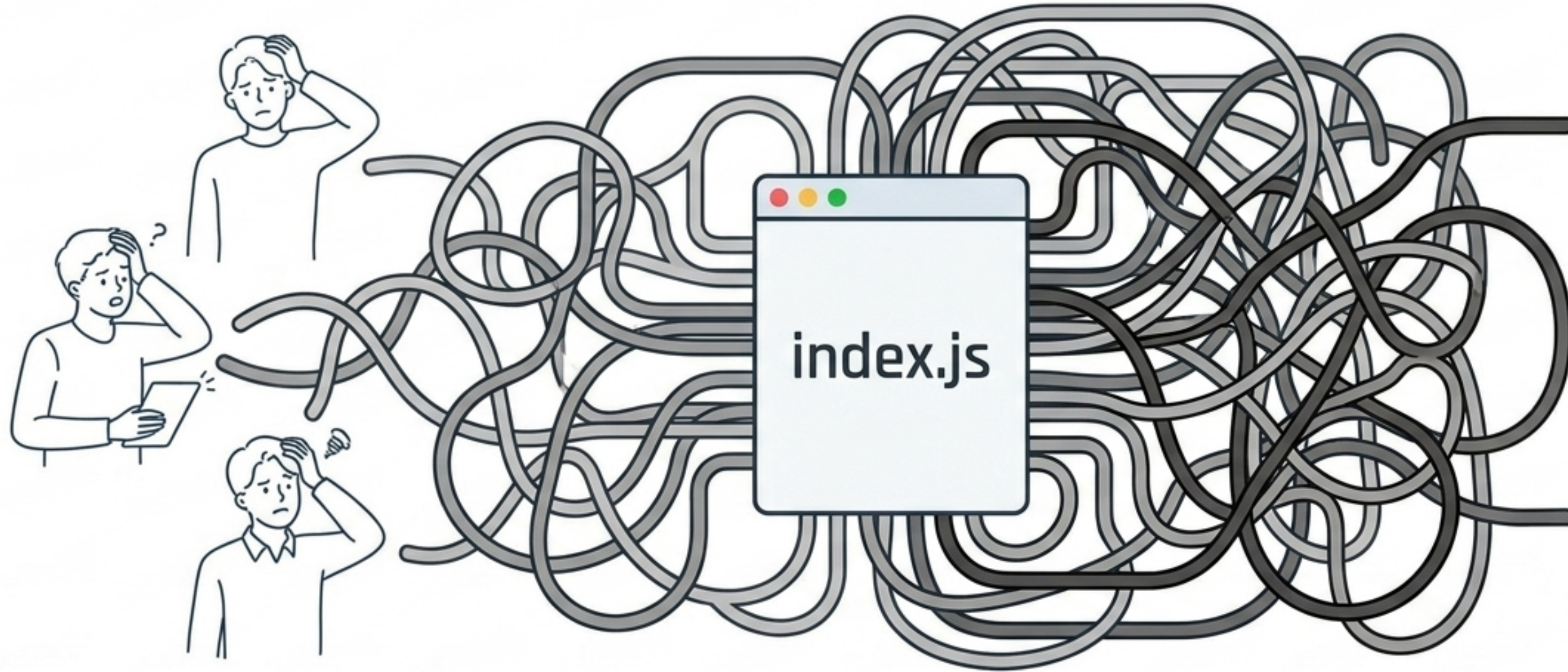
Janela de Código 1 (PUT):

```
app.put('/users/:id', (req, res) => {
  const id = req.params.id;
  const index = users.findIndex(u => u.id === id);
  if (index !== -1) {
    users[index] = req.body;
    res.send('Usuário atualizado com sucesso!');
  } else {
    res.status(404).send('Usuário não encontrado');
  }
});
```

Janela de Código 2 (DELETE):

```
app.delete('/users/:id', (req, res) => {
  const id = req.params.id;
  users = users.filter(u => u.id !== id);
  res.send('Usuário excluído com sucesso!');
});
```

O Problema do Monolito de Rotas



À medida que a API cresce, concentrar todos os manipuladores de rotas (Users, Produtos, Pedidos) em um único arquivo **index.js** torna a manutenção insustentável, gerando conflitos de equipe e código ilegível.

Arquitetura Modular com express.Router()

Antes: O Módulo

project/
└ routes/
 └ **users.js**

```
const express = require('express');  
const router = express.Router();  
  
// rotas aqui  
  
module.exports = router;
```

O Módulo: No arquivo routes/users.js, usamos `const router = express.Router()` e exportamos o roteador no final.

Depois: A Importação

```
const userRoutes = require('./routes/users');  
  
app.use('/users', userRoutes);
```

A Importação: O index.js fica limpo, apenas delegando responsabilidades para os respectivos roteadores.

Referências Bibliográficas

BROWN, Ethan. Programação web com Node e Express: Beneficiando-se da Stack JavaScript. Editora Novatec, 2020.

DUARTE, Luiz. Programação Web com Node.js: Completo, do Front-end ao Back-end. LuizTools, Edição do Kindle.

EXPRESS.JS. Documentação Oficial do Express.js. Disponível em: [<http://expressjs.com/pt-br/>](http://expressjs.com/pt-br/).

FIELDING, Roy Thomas. Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado em Filosofia) - University of California, Irvine, 2000.

MDN WEB DOCS. HATEOAS - MDN Web Docs Glossary. Disponível em: [<https://developer.mozilla.org/>](https://developer.mozilla.org/).